159102 workshop – Week 10 – Some selected solutions

1. Write a C++ program that reads positive real numbers into a vector (in a random order). Use a trailing value of -1. Sort the numbers into ascending order and display the sorted numbers. Your program must include the following two functions:

```
void sort(vector<float> & v);
void display(vector<float> & v);
```

Solution to Q1:

```
#include <iostream>
#include <vector>
using namespace std;

vector<float> numbers;

void sort(vector<float> & v);
void display(vector<float> & v);

int main() {
  float temp;
  temp = 99.9;  // just to start the loop
  while (temp > 0.0) {
    cout << "Enter a positive real number (-1 to stop) ";
    cin >> temp;
    if (temp > 0.0) {
      numbers.push_back(temp);
    }
  }
  sort(numbers);
  display(numbers);
}

void sort(vector<float> & v) {
  int total, pass, i;
  float temp;
  total = v.size();
  for (pass = 0; pass < total; pass++) {
    for (i = 0; i < total - 1; i++) {
      if (v[i] > v[i+1]) {
        temp = v[i];
        v[i] = v[i+1];
        v[i+1] = temp;
      }
    }
  }
}

void display(vector<float> & v) {
  int total, i;
  total = numbers.size();
  for (i = 0; i < total; i++) {
    cout << numbers[i] << ", ";
  }
```

```
  cout << endl;
}
```

2. In your sorting program above, change the first line of the `sort` function to be:

```
void sort(vector<float> v);
```

(*Note that you will also have to change the first line in the function definition*.)
Do not change any other code. Run the program. Explain what happens.

Solution to Q2:

The vector does not get sorted – this is shown by the display. This happens because the sorting needs to move data around in the vector, i.e. the vector **needs to be changed**. If you want to change a parameter then you must use call-by-reference to pass the parameter to the function. By removing the & symbol, we have stopped the parameter being call-by-reference.

Extra note: there is no need for the parameter to display to be call-by-reference. However, many programmers routinely use call-by-reference for large data structures because this means that only a pointer is passed across to the function. If you remove the & (convert it to call-by-value) then the entire data structure will be copied across to the function and this wastes time.

3. Discuss what you would need to do to change your sorting program above so that it uses an array and not a vector. What is the single biggest difference between the two programs? This is for discussion only – there is no need to actually write the program using an array.

Solution to Q3:

Using an array would be very similar to using a vector. You would need to define the array:

```
float numbers[500];
```

The single biggest difference is that you need to guess the maximum size of the array. In the example above, the guess for the maximum size is 500. This may not be big enough...

4. The program below enters data for bank accounts and then searches to find a particular bank account. However, the program is not working because there are 6 errors in it. Identify the errors and state what needs to be done to correct each error.

```
#include <iostream>
#include <cstdio>  // for getchar
#include <vector>
using namespace std;

class bank_account {
private:
  int accnumber;
  string name;
  float balance;
```

```
public:
  void loaddata();
  void display();
};

void enteraccounts();

int main() {
  int search, total, num, i;
  enteraccounts();
  cout << "Which account do you want to display? ";
  cin >> search;
  total = accounts.size();
  for (i = 0; i < total; i++) {
    num = accounts[i].getnum();
    if (num != search) {
      accounts[i].display();
    }
  }
}

void enteraccounts() {
  // enter 3 accounts - this could be changed
  int i;
  bank_account temp;
  for (i = 0; i < 3; i++) {
    temp.loaddata();
  }
}

//--------------- methods for the bank_account ------------
void loaddata() {
  cout << "Enter the bank account number ";
  cin >> accnumber;
  cout << "Enter the name of the account ";
  getline(cin, name);
  cout << "Enter the balance, e.g. 32.45 ";
  cin >> balance;
}

void bank_account::display() {
  cout << "Data for Account " << accnumber << endl;
  cout << "Name: " << name << endl;
  cout << "Balance: $" << balance << endl << endl;
}
```

Solution to Q4:

1) method missing from the class, we need:

```
int getnum() { return accnumber; }
```

2) vector has not been defined, we need:

```
vector<bank_account> accounts;
```

3) incorrect test inside the if statement, it should be:

```
if (num == search) {
```

4) inside “enteraccounts”, data is not actually placed in the vector. Insert new line:

```
temp.loaddata();
accounts.push_back(temp);
```

5) method must have the class name before the function name, as follows:

```
void bank_account::loaddata() {
```

6) in method “loaddata”, entering a number (accnumber) before a string (name) will cause no data to be entered for the string. Need to flush the input buffer with an extra line as follows:

```
cin >> accnumber;
getchar();
```

5. Assume that the program below has a function to display all items in the stack (similar to the “display and empty” function in the notes). Write down what values would be displayed and in what order.

```
stack<int> s;

int main() {
  s.push(27);
  s.push(83);
  s.pop();
  s.push(56);
  -- call function here to display all items
}
```

Solution to Q5:

It would display 56 and then 27. 56 is the last item placed on the stack so it is the first item displayed. 83 was placed on the stack but was then deleted by using pop.